

Engenharia de Requisitos

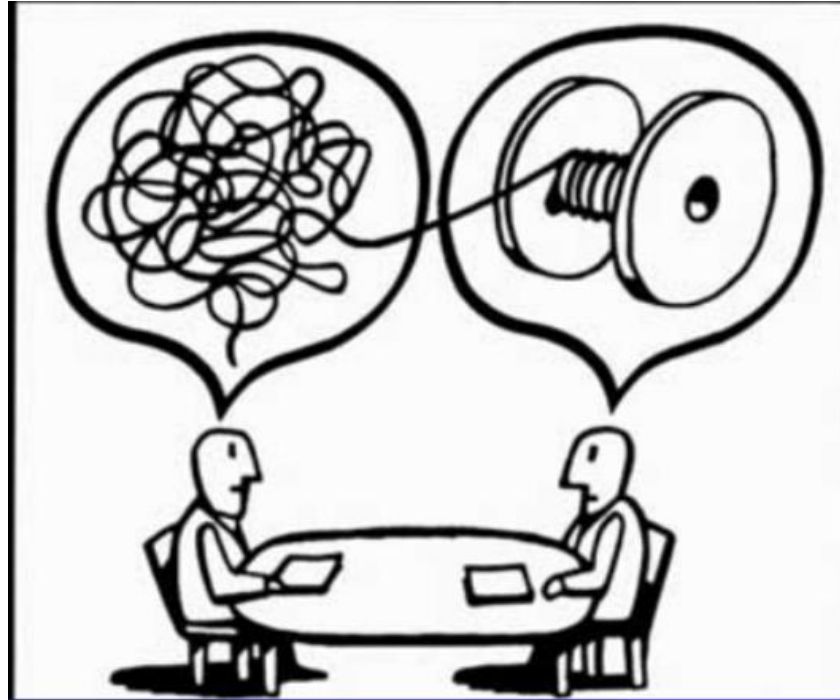
O que?

- .Estabelecer os serviços que um cliente requer de um sistema e as restrições sob as quais ele opera e é desenvolvido**
- .Os requisitos do sistema são descrições dos serviços e restrições que são geradas no processo de engenharia de requisitos**

Por que?

- A experiência de desenvolvimento nos diz que este é o documento mais importante do sistema, ele que de fato especifica o que será feito
- Mesmo em processos iterativos, possuir um documento com o conjunto final de requisitos é fundamental para entender o sistema e possibilitar sua manutenção

Problema Central: Comunicação



O que é um requisito?

- Requisitos podem variar desde uma descrição abstrata de alto nível de um serviço ou restrição do sistema até uma função matemática especificada detalhadamente
- Isto ocorre porque os requisitos têm uma função dual
 - Podem ser a base para um lance para um contrato – portanto devem ser abertas a interpretação
 - Podem ser a base para o próprio contrato – portanto devem ser definidas em detalhes

Abstração de Requerimentos (Davis)

“Se uma empresa precisa desenvolver um contrato para um grande sistema de software, deve declarar suas necessidades de forma suficientemente abstrata para que a solução não seja pré-definida. Assim diversos proponentes podem oferecer formas distintas de satisfazer as necessidades da empresa. Uma vez que o contrato tenha sido definido, o proponente precisa escrever uma definição do sistema mais precisa para que o cliente possa entender e validar o que o software vai fazer. Ambos os documentos podem ser chamados de documento de requisitos do sistema”

Tipos de Requisitos

•Requisitos do Usuário

–Afirmações em linguagem natural e diagramas dos serviços que o software oferece e suas restrições operacionais. Escrito para os clientes

•Requisitos do Sistema

–Um documento estruturado com descrições detalhadas das funções, serviços e restrições operacionais do sistema. Define o que deve ser implementado e pode ser parte do contrato entre o cliente e o desenvolvedor

Exemplo: Requisito de Usuário

.O sistema deve gerar relatórios gerenciais mensais que mostrem o custo dos medicamentos prescritos por cada clínica durante aquele mês

Exemplo: Requisito de Sistema

- No último dia útil de cada mês deve ser gerado um resumo dos medicamentos prescritos, seus custos e as prescrições de cada clínica.
- Após 17:30 do último dia útil do mês, o sistema deve gerar automaticamente o relatório para impressão.
- Um relatório será criado para cada clínica, listando os nomes dos medicamentos, o número total de prescrições, o número de doses prescritas e o custo total dos medicamentos prescritos.
- Se os medicamentos estão disponíveis em diferentes unidades de dosagem (e.g. 10mg e 20mg), devem ser criados relatórios separados para cada unidade.

Para quem?

•Requisitos de Usuário

- Gerentes clientes
- Usuários finais
- Engenheiros clientes
- Gerentes contratantes
- Arquitetos de sistema

•Requisitos de Sistema

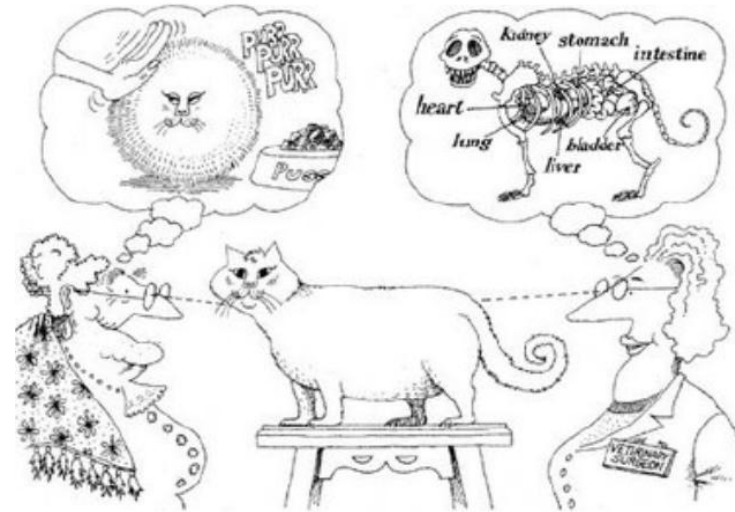
- Usuários finais
- Engenheiros clientes
- Arquitetos de sistema
- Desenvolvedores

Stakeholders

.Qualquer pessoa ou organização que é afetada pelo sistema de alguma forma e que tem um interesse legítimo

.Tipos:

- Usuário final
- Gerentes do sistema
- Donos do sistema
- Externos



Exemplo: Stakeholders no Sistema Médico

- .Pacientes
- .Médicos
- .Enfermeiras
- .Recepcionistas
- .Funcionários de TI
- .Órgãos de regulação
- .Gerentes das clínicas
- .Gerentes de informações médicas

Requisitos em Métodos Ágeis

- Muitos destes métodos preconizam que fazer um documento detalhado é perda de tempo, pois requisitos mudam com frequência
- Os requisitos são incrementais expressados através dos Cenários
- É prático para sistemas comerciais mas problemático para sistemas que exigem uma análise prévia (sistemas críticos) ou que são desenvolvidos por várias equipes

Requisitos Funcionais e Não-Funcionais

Classificação de Requisitos

.Funcionais

- Serviços que o sistema deve prover, como o sistema deve reagir a certas entradas particulares e como o sistema deve se comportar em situações particulares
- Também podem dizer o que o sistema não deve fazer

.Não-funcionais

- Restrições sobre os serviços oferecidos pelo sistema como tempo de resposta, processo de desenvolvimento, padrões
- Geralmente se aplicam ao sistema como um todo e não em características específicas

Requisitos Funcionais

- Descreve a funcionalidade ou funções do sistema
- Depende do tipo de software, usuários e do tipo de sistema onde o software é usado
- Requisitos funcionais de usuários podem ser descrições de alto nível do que o sistema faz
- Requisitos funcionais de sistema devem descrever o serviço em detalhes

Exemplo: Requisitos Funcionais

- Um usuário deve ser capaz de buscar a lista de agendamentos de todas as clínicas
- O sistema deve gerar a cada dia, para cada clínica, uma lista dos pacientes agendados
- Cada usuário do sistema deve ser identificado unicamente por seu código funcional de 8 dígitos

Imprecisão

- Podem surgir problemas quando os requisitos não são enunciados de forma precisa
- Descrições ambíguas podem ser interpretadas de formas distintas por desenvolvedores e usuários
- Considere o termo “buscar” no exemplo anterior
 - Intenção do usuário: buscar o nome do paciente em todas as listas de agendamento de todas as clínicas
 - Interpretação do Desenvolvedor: buscar o nome do paciente em uma clínica individual. O usuário escolhe qualquer uma das clínicas e depois busca

Exemplo: SIGAA

- Listar todos os alunos de todos os períodos do curso X aptos a cursar a disciplina Y
- Intenção: listar todos os alunos de um determinado curso que têm os pré-requisitos para cursar determinada disciplina.
- Implementação: Escolhe o período do aluno, depois faz a busca

Completude e Consistência

- Em princípio, os requisitos devem ser completos e consistentes

- Completo

- Descreve todas as funcionalidades requeridas

- Consistente

- Não deve haver conflitos ou contradições entre as descrições

- Na prática, quase nunca é

Exemplo: Inconsistência

- Caso a senha seja esquecida é necessário trocá-la.
- Para trocar a senha é necessário abrir um chamado.
- Para abrir um chamado é necessário acessar o sistema.

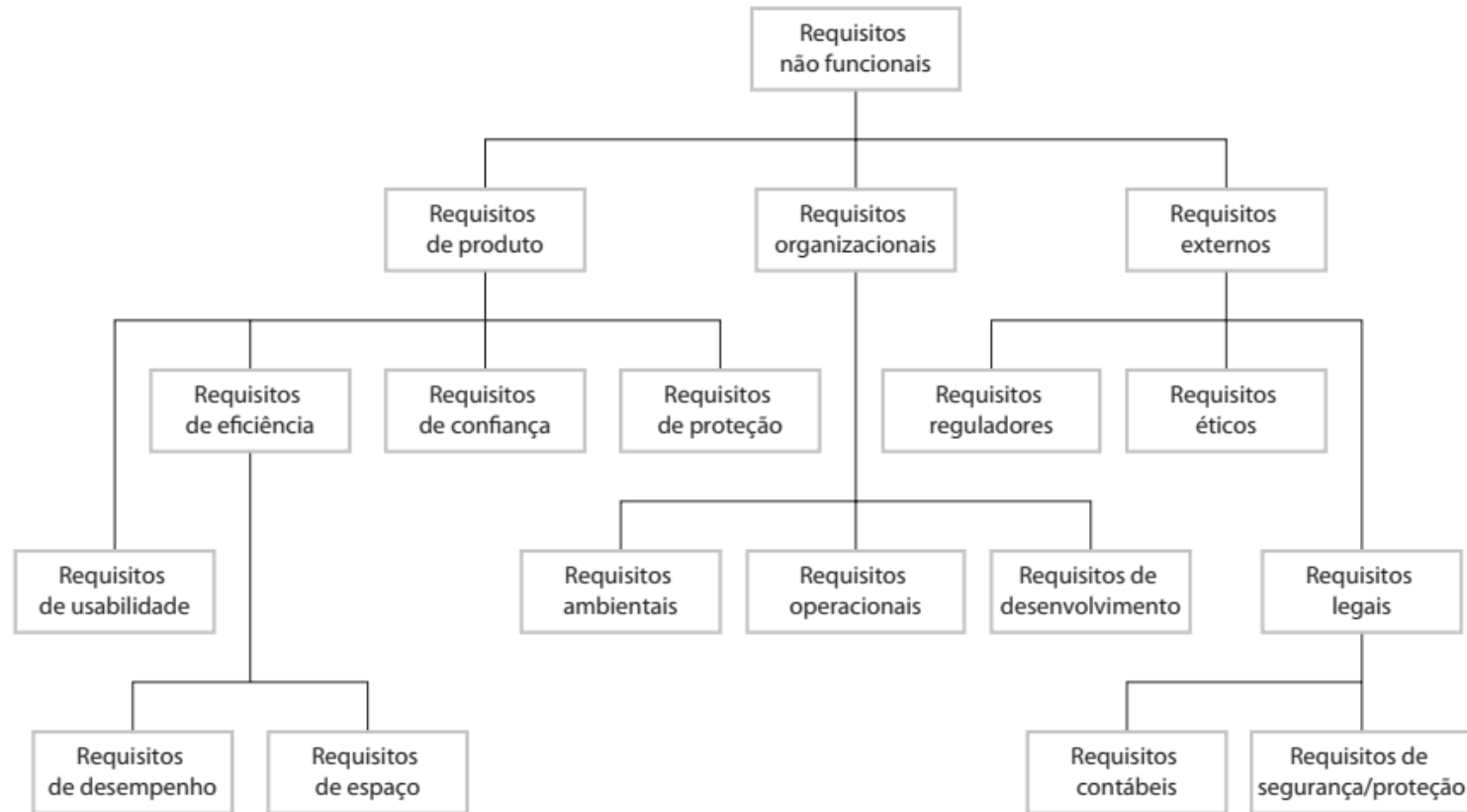
Requisitos não-funcionais

- Definem propriedades e restrições do sistema
 - Confiabilidade, tempo de resposta, uso de memória, dispositivos de entrada e saída, etc.
- Requisitos de processo também podem especificar uma determinada IDE, linguagem de programação ou método de desenvolvimento
- Às vezes os requisitos não-funcionais são mais críticos que os funcionais. Se não são atendidos, o sistema pode ser inútil

Vamos pensar...

.Pelo que foi descrito, você consegue pensar em requisitos não funcionais que são cruciais para a utilidade do sistema? Dê exemplos.

Tipos de requisitos não-funcionais



Implementação de requisitos não-funcionais

- Podem afetar a arquitetura geral do sistema

- Por exemplo, minimizar comunicação para atender requisitos de desempenho

- Um único requisito não-funciona, como de segurança, pode gerar vários requisitos funcionais relacionados que definem os serviços necessários

- Às vezes gera requisitos que restringem outros já existentes

Exemplo: Requisitos não-funcionais

.Requisitos de produto

- O sistema deve estar disponível para todas as clínicas no horário comercial. Dentro do horário comercial, o *downtime* (tempo indisponível) não deve exceder cinco segundos em um dia.

.Requisitos Organizacionais

- Usuários devem autenticar-se usando seu cartão de identidade funcional

.Requisitos Externos

- O sistema deve implementar os padrões de privacidade vigentes para dados médicos

Objetivos e Requisitos

- Pode ser difícil enunciar os requisitos não-funcionais de forma precisa
- Enunciados imprecisos são difíceis de verificar
- Objetivo
 - Intenção geral do usuário
- Requisito não-funcional verificável
 - Usa algum critério que pode ser objetivamente testado

Exemplos de Ambiguidade

- A janela deve abrir rapidamente
- O sistema deve ser flexível
- O cálculo deve ser eficiente
- A interface deve ser melhor que a atual
- Não devem ser mostradas muitas mensagens de erro
- A exibição do mapa de navegação deve ser amigável

Exemplo

.Objetivo

- O sistema deve ser fácil de utilizar pela equipe médica e organizado de modo a minimizar erros

.Requisito não-funcional verificável

- A equipe médica deve ser capaz de usar todas as funcionalidades do sistema após um treinamento de quatro horas. Após este treino, o número médio de erros cometidos por um usuário não deve exceder dois por hora de uso do sistema.

Métricas Comumente Usadas

.Velocidade

- Transações por segundo, tempo de resposta, taxa de atualização

.Tamanho

- MBytes, número de chips ROM

.Facilidade de uso

- Tempo de treino, número de telas de ajuda

.Confiabilidade

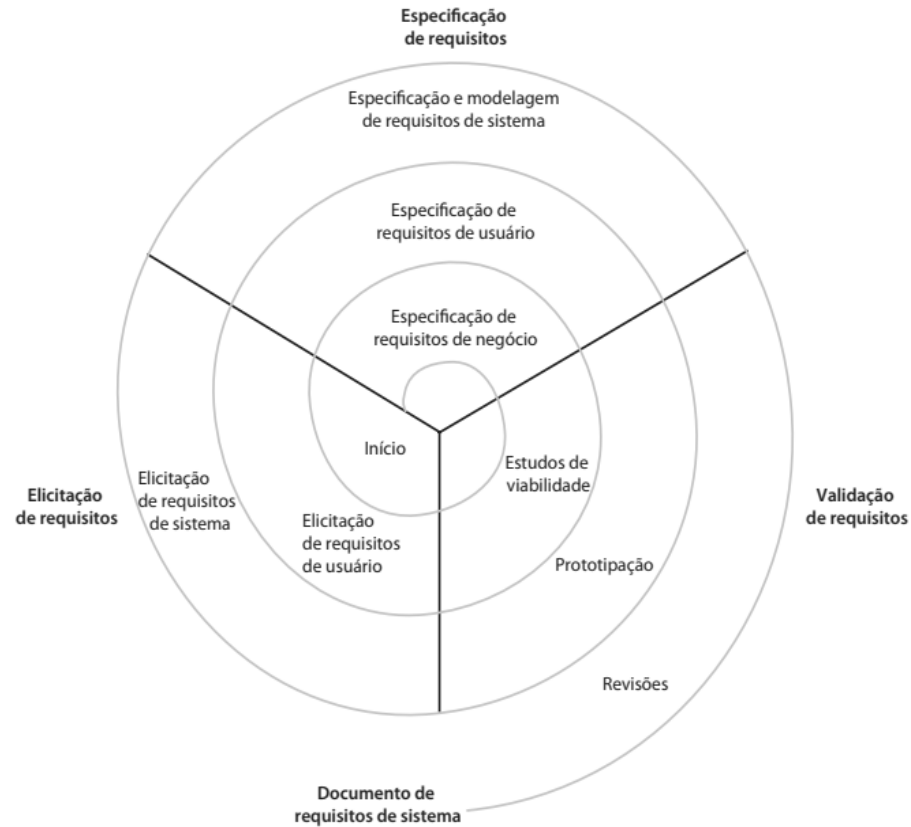
- Tempo médio até a falha, probabilidade de indisponibilidade, taxa de ocorrência de falhas, disponibilidade

Processos

Processos

- Os processos variam amplamente a depender do domínio da aplicação, das pessoas envolvidas e da organização que desenvolve os requisitos
- Algumas atividades são comuns a todos os processos
 - Viabilidade
 - Elicitação e Análise
 - Especificação
 - Validação
- Na prática, essas atividades são intercaladas

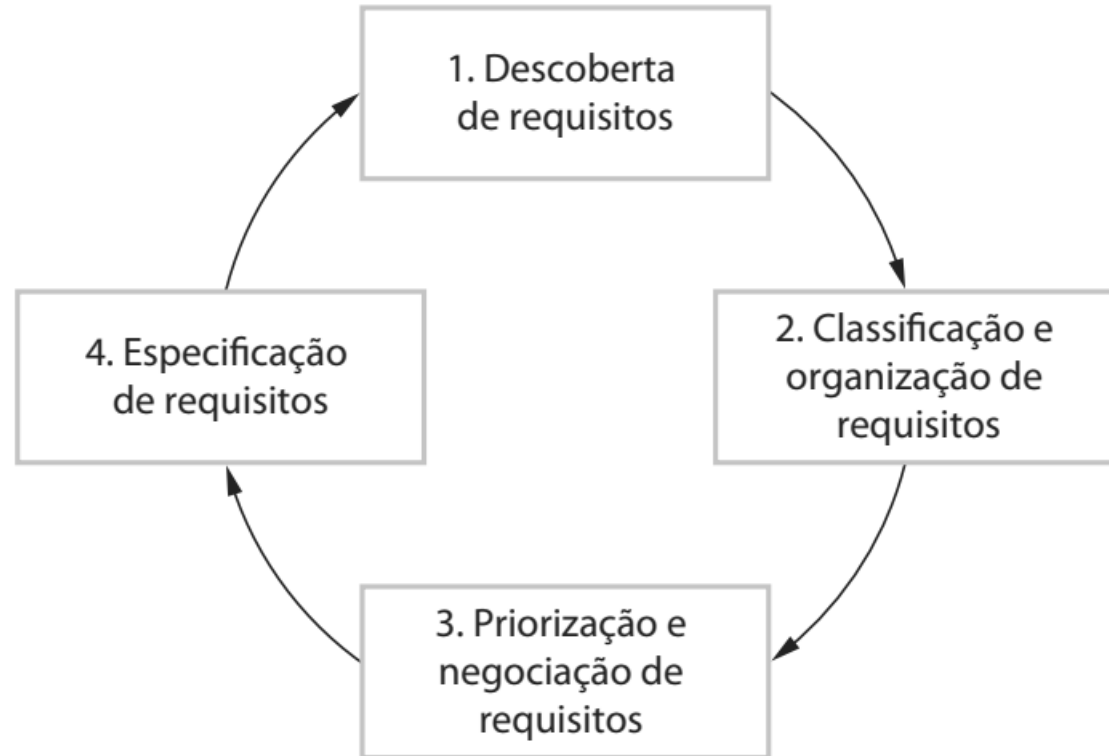
Visão em espiral



Elicitação de Requisitos

- Também chamado de descoberta de requisitos
- Equipe técnica em conjunto com o cliente para entender o domínio da aplicação, os serviços que o sistema deve prover e as restrições operacionais
- Pode envolver usuários finais, gerentes, engenheiros de manutenção, especialistas no domínio... Todos os stakeholders

Etapas



Problemas

- .Stakeholders não sabem o que realmente querem
- .Stakeholders se expressam em seu próprio jargão
- .Stakeholders distintos têm requisitos conflitantes
- .Políticas organizacionais influenciam os requisitos
- .Os requisitos mudam no processo de análise. Novos stakeholders surgem e o ambiente do negócio muda

Descoberta de Requisitos

- É o processo de obter informações sobre o sistema desejado e sistemas existentes e distilar os requisitos de usuário e de sistema a partir destas informações
- Interação com stakeholders, de gerentes a reguladores externos
- A maioria dos sistemas tem vários stakeholders

Entrevistas

•Entrevistas formais ou informais com stakeholders são parte dos processos de Engenharia de Requisitos

•Tipos de Entrevista

- Entrevistas fechadas, baseadas em questões pré-determinadas
- Entrevistas abertas, exploratórias

Entrevistas

.Entrevistando de forma eficiente

- Tenha a mente aberta, evite ideias preconcebidas sobre requisitos e esteja disposto a ouvir
- Provoque o entrevistado a continuar a discussão usando perguntas chave, propostas de requerimento ou trabalhando juntos num protótipo

Problemas com as Entrevistas

- Especialistas em um domínio podem usar uma linguagem para descrever seu trabalho que não é fácil de ser plenamente compreendida por pessoas de fora da área
- Não são eficientes para identificar requisitos de domínio
 - Problemas de comunicação
 - Dentro de um domínio, alguns pontos são tão “óbvios” que as pessoas têm dificuldade de explicar de forma articulada ou acham que não vale a pena citar já que “todo mundo sabe”

Etnografia

- .Técnica de observação usada para compreender os processos operacionais e ajudar a extrair requisitos de apoio para esses processos.
- .Um analista observa o dia a dia do trabalho e faz anotações sobre as tarefas reais em que os participantes estão envolvidos.

Etnografia

.Eficaz para descobrir:

–Requisitos derivados de como as pessoas realmente trabalham, e não da forma como dizem que deveriam trabalhar.

–Requisitos derivados da cooperação e conhecimento das atividades de outras pessoas.

.Entende o processo atual mas não é capaz de identificar novas características que devem ser acrescentadas ao sistema

Etnografia

- A etnografia pode ser usada com a prototipação:
 - As informações da etnografia ajuda a fazer menos iterações do protótipo
 - As informações do protótipo ajudam a focar a etnografia e definir o que precisa ser melhor observado em uma próxima etapa

Cenários

- Como na XP, descrevem situações reais de uso
- As pessoas em geral acham mais fácil se relacionar com exemplos de uma situação real que com descrições abstratas. São mais dispostas a debater a situação.
- Deste debate podemos detalhar os requisitos

Cenários

- Uma descrição estruturada de uma interação com o sistema
- Deve incluir:
 - Uma descrição da situação inicial
 - Uma descrição do fluxo normal de eventos
 - Uma descrição do que pode dar errado
 - Informações sobre outras atividades concorrentes
 - Uma descrição da situação quando o cenário termina

Exemplo: Cenário de Coleta de Histórico

.Suposição Inicial:

–Um paciente é atendido em uma clínica por uma recepcionista, ela gera um registro no sistema e coleta suas informações pessoais. Uma enfermeira conectada ao sistema coleta o histórico médico do paciente.

.Normal:

–A enfermeira busca o paciente pelo nome, se houver mais de um com o mesmo nome, usa-se também a data de nascimento. A enfermeira escolhe a opção do menu para adicionar o histórico médico. A enfermeira segue, então, uma série de telas do sistema para inserir informações sobre consultas em outros locais, condições médicas (seleciona de um menu), medicação atual (seleciona de um menu), alergias (texto livre) e

Exemplo: Cenário de Coleta de Histórico

.O que pode dar errado:

–Prontuário não existe ou não pode ser encontrado. A enfermeira deve criar um novo registro e registrar as informações pessoais. As condições do paciente ou a medicação não estão entre os itens do menu. A enfermeira deve escolher a opção “outros” e inserir texto livre. O paciente não fornece informações sobre seu histórico médico. A enfermeira deve inserir texto livre registrando o fato.

Exemplo: Cenário de Coleta de Histórico

.Outras atividades:

- Enquanto a informação está sendo inserida, o registro pode ser consultado, mas não editado por outros agentes

.Estado do sistema na conclusão:

- Usuário está conectado. Prontuário, incluindo histórico médico, é inserido no banco de dados e um registro é adicionado ao *log* do sistema, mostrando o tempo de início e fim da sessão e a enfermeira envolvida.

Especificação de Requisitos

Especificação

- Escrita dos requisitos de usuário e de sistema em um documento
- Requisitos de usuários têm que ser entendidos por pessoas sem conhecimento técnico
- Requisitos de sistema são mais detalhados e podem incluir mais informações técnicas

Requisitos e Projeto

- Requisitos deveriam dizer “o que” o sistema faz e não “como” ele faz
- Na prática, requisitos e projeto são inseparáveis
 - Uma arquitetura pode ser definida para atender os requisitos não funcionais
 - O sistema pode operar com outros sistemas que geram requisitos de projeto
 - Regulações e políticas externas e organizacionais podem impor um tipo de projeto específico

Formas de Escrever

.Linguagem Natural

- Escritos usando frases numeradas. Cada frase deve expressar um requisito

.Linguagem Natural Estruturada

- Usam uma formatação padrão, como um formulário. Cada campo dá informações sobre um aspecto do requisito

.Linguagem de Descrição de Projeto

- Similar a uma linguagem de programação, mas mais abstrata. Pouco usada atualmente, mas pode ser útil para descrever interfaces

Linguagem Natural

.É a mais fácil de ser entendida por todos os envolvidos no desenvolvimento

.Também é a mais fácil de ser mal interpretada, pois é muitas vezes ambígua

Recomendações para a Escrita

- Invente um formato padrão e siga para todos os requisitos
- Use a língua de forma correta. Por exemplo, use “deve” para requisitos obrigatórios e “deveria” para requisitos desejáveis
- Use destaques para identificar partes essenciais do requisito
- Evite jargão técnico
- Inclua uma explicação de porque o requisito é necessário

Problemas da Linguagem Natural

- Falta de clareza

- Ser preciso é difícil sem deixar a frase muito longa ou difícil de entender

- Confusão de requisitos

- Requisitos funcionais e não-funcionais tendem a se misturar

- Agrupamento de requisitos

- Vários requisitos distintos podem ser expressos juntos

Exemplo: Bomba de Insulina

- O sistema deve medir o açúcar no sangue e fornecer insulina, se necessário, a cada 10 minutos. (Alterações no índice glicêmico são relativamente lentas então aferição mais frequente é desnecessária; medidas menos frequentes poderiam levar a níveis desnecessariamente altos)
- O sistema deve executar um auto-teste a cada minuto com as condições a serem testadas e as ações associadas definidas na Tabela 1. (Um auto-teste pode descobrir falhas de hardware e software e alertar o usuário de que operação normal pode não ser possível)

Especificações Estruturadas

- As características dos requisitos são padronizadas
- Menos liberdade para quem escreve
- Funciona bem para alguns sistemas, como embarcados
- Às vezes é muito restritiva para sistemas comerciais

Formulário de Especificação

- .Definição da função ou entidade
- .Descrição das entradas e de onde vêm
- .Descrição das saídas e para onde vão
- .Descrição das informações necessárias para a computação e outras entidades usadas
- .Descrição da ação a ser tomada
- .Pre e pós condições (se apropriado)
- .Efeitos colaterais (caso existam)

Exemplo: Bomba de Insulina

.Função

- Calcula dose de insulina: nível seguro de açúcar

.Descrição

- Calcula a dose de insulina a ser fornecida quando o nível de açúcar está na zona de segurança entre três e sete unidades

.Entradas

- Leitura atual da glicemia (r_2), duas leituras anteriores (r_1 e r_0)

.Fonte

Leitura atual da glicemia pelo sensor. Outras leituras da memória

Exemplo: Bomba de Insulina

.Saída

- A dose de insulina a ser administrada (CompDose)

.Destino

- Laço de controle principal

.Ação

- CompDose é zero se o nível de açúcar está estável ou em queda ou se o nível está aumentando, mas a taxa de aumento está diminuindo. Se o nível está aumentando e a taxa de aumento está aumentando, então CompDose é calculado dividindo-se a diferença entre o nível atual e o nível anterior por quatro e arredondando-se o resultado. Se o resultado é

Exemplo: Bomba de Insulina

.Requisitos

- Duas leituras anteriores, para que a taxa de variação do nível possa ser calculada

.Pré-condição

- O reservatório de insulina contém, no mínimo, a dose única máxima permitida

.Pós-condição

- r_0 é substituída por r_1 , r_1 é substituída por r_2

Tabelas

- Você pode adicionar informações extras para facilitar a descrição
- Especialmente útil quando há várias situações alternativas possíveis

Modificando o Exemplo Anterior

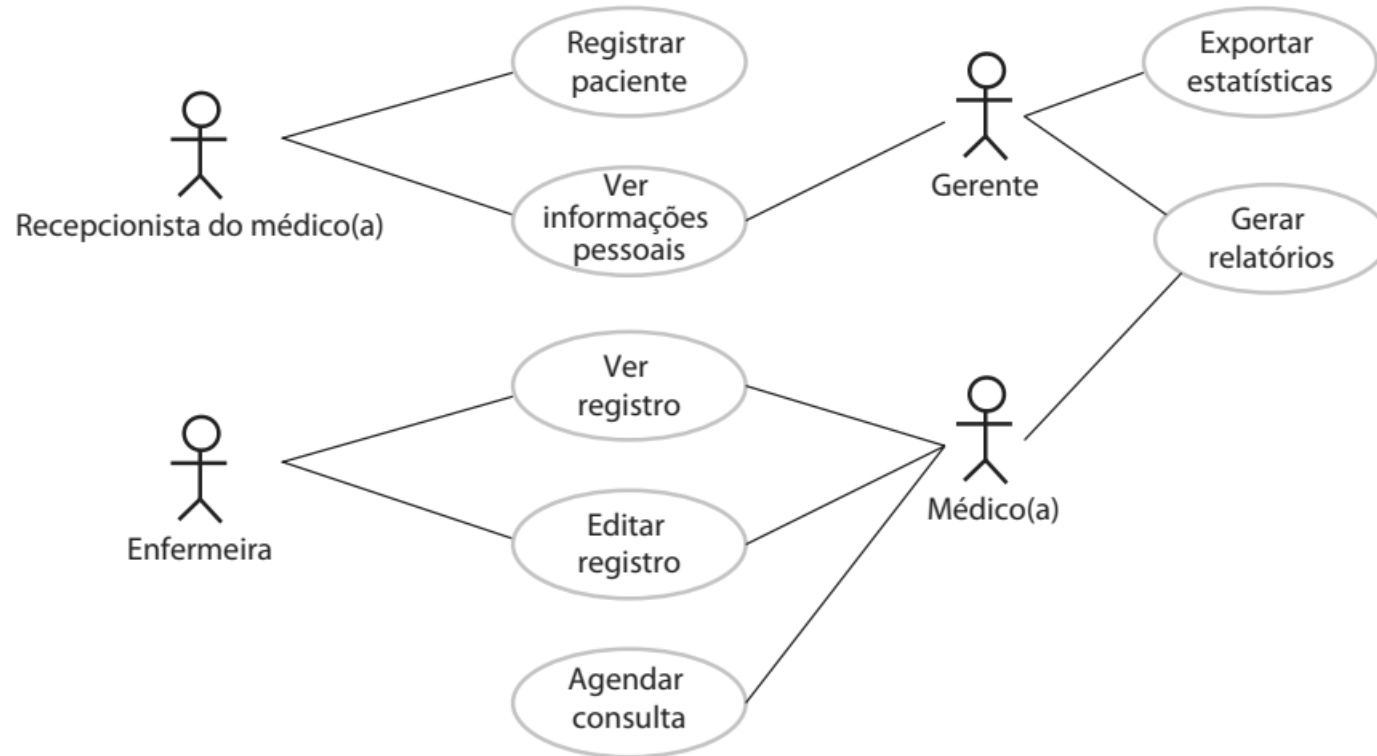
.Ação

Condição	Ação
Nível caindo ($r_2 < r_1$)	CompDose = 0
Nível estável ($r_e = r_1$)	CompDose = 0
Nível subindo e taxa de aumento decrescente ($(r_2 - r_1) < (r_1 - r_0)$)	CompDose = 0
Nível subindo e taxa de aumento estável ou crescente ($(r_2 - r_1) \geq (r_1 - r_0)$)	$X = \text{round}((r_2 - r_1)/4)$ Se X é 0 Então CompDose = DoseMinima Senão CompDose = X

Casos de Uso

- Descrevem as possíveis interações, geralmente através de um diagrama da UML (Linguagem de Modelagem)
- Atores, que podem ser pessoas ou outros sistemas, são representados por figuras “palito”.
- Classes de interação são representadas por elipses
- Linhas ligam os atores às interações, flechas podem ser adicionadas para indicar quem inicia a ação
- Cada caso de uso necessita de uma descrição textual

Exemplo



Exemplo: Agendar Consulta

.Agendar a consulta permite que dois ou mais médicos de consultórios diferentes possam ler o mesmo registro ao mesmo tempo. Um médico deve escolher, em um menu de lista de médicos online, as pessoas envolvidas. O prontuário do paciente é então exibido em suas telas, mas apenas o primeiro pode editar o registro. Além disso uma janela de mensagens de texto é criada para ajudar a coordenar as ações. Opcionalmente, a videoconferência pode ser ativada na janela de mensagens.

O Documento de Requisitos

- É a declaração oficial do que é pedido dos desenvolvedores do sistema
- Deve incluir definições dos requisitos de usuário e de sistema
- Não é um documento de projeto, na medida do possível deve dizer “o que” faz e não “como” faz

Usuários do Documento

•Clientes

- Especificam os requisitos e verificam se o documento reflete suas necessidades

•Gerentes

- Estimar custos e fazer o planejamento do desenvolvimento

•Engenheiros de Sistema

- Entender qual sistema deve ser desenvolvido

•Engenheiros de Teste

Desenvolver testes de validação para o sistema

Estrutura do Documento (IEEE)

.Prefácio

–Define os possíveis leitores e descreve o histórico de versões, incluindo uma justificativa para a nova versão e um resumo das mudanças em cada versão

.Introdução

–Descreve a necessidade para o sistema. Descrição breve das funções e de como ele funciona com outros sistemas. Diz como o sistema atende os objetivos globais do cliente

.Glossário

Define termos técnicos utilizados

Estrutura do Documento (IEEE)

•Definição de Requisitos de Usuário

- Descreve os requisitos funcionais e não-funcionais. Normas de produtos e processos que devem ser seguidos devem ser especificados

•Arquitetura do Sistema

- Deve apresentar uma visão geral em alto nível da arquitetura do sistema previsto, mostrando a distribuição de funções entre os módulos do sistema. Destacar componentes reusados.

•Especificação de Requisitos de Sistema

- Detalhamento dos requisitos funcionais e não-funcionais. Interfaces com outros sistemas podem ser definidas

Estrutura do Documento (IEEE)

.Modelos do Sistema

- Modelos gráficos do sistema mostrando o relacionamento entre os componentes e o ambiente**

.Evolução do Sistema

- Descreve mudanças previstas em função da evolução de hardware ou de mudanças nas necessidades do usuário**

.Apêndices

- Informações extras como requisitos de hardware e banco de dados**

.Índice

Validação de Requisitos

Por que?

- Garantir que os requisitos definem o sistema que o cliente realmente quer
- Corrigir um erro de requisito após a entrega pode custar centenas de vezes mais que corrigir um erro de implementação

Checklist

.Validade

–O sistema fornece as funções que melhor atendem as necessidades do cliente?

.Consistência

–Existem requisitos em conflito?

.Compleitude

–Todas as funcionalidades que o cliente solicitou estão incluídas?

.Realismo

As funcionalidades podem ser implementadas com o orçamento e

Técnicas de Validação

.Revisão dos requisitos

- Análise manual sistemática dos requisitos

.Prototipação

- Usar um modelo executável do sistema para verificar os requisitos

.Geração de casos de teste

- Desenvolve situações de teste para verificar que os requisitos foram atendidos

Revisões de Requisitos

- .Devem ser feitas desde a definição dos requisitos
- .Toda a equipe e stakeholders deve estar envolvida
- .Pode ser formal (com documentos completos) ou informal

- .Uma boa comunicação entre desenvolvedores e stakeholders pode resolver problemas no início!

Checklist da Revisão

.Verificabilidade

- O requisito é realisticamente testável?

.Compreensibilidade

- O requisito foi adequadamente entendido?

.Rastreabilidade

- A origem do requisito está claramente definida?

.Adaptabilidade

- O requisito pode ser modificado sem grande impacto nos demais?

Mudanças nos Requisitos

Alterações são Inevitáveis

- O ambiente técnico e comercial do sistema sempre muda com o tempo
 - Novos hardwares, novos sistemas, prioridades de negócio, legislações
- Quem paga pelo sistema e os usuários nem sempre são as mesmas pessoas
 - Isso implica que os requisitos de uso real nem sempre são levados em conta. Além disso, usuários distintos têm prioridades distintas, que podem ser conflitantes ou contraditórias

Evolução de Requisitos

- A evolução dos requisitos é um reflexo dessas mudanças
- A evolução dos requisitos em geral guia a evolução do software
- Acontece tanto durante o desenvolvimento quanto após a entrega

Gerenciamento de Requisitos

- É o processo de gerenciar esses requisitos dinâmicos durante o desenvolvimento
- É necessário rastrear requisitos individuais e manter conexões entre requisitos dependentes
 - Facilita entender o impacto das mudanças
- É necessário estabelecer um processo formal para propor mudanças e sempre relacionar com os requisitos de sistema

Decidindo se uma Alteração deve ser Aceita

.Análise do problema e Especificação da alteração

- Verificar se o problema é válido e se a alteração o resolve. Se negado é devolvido ao solicitante que pode ser mais específico ou remover a proposta

.Análise dos Custos da Alteração

- Verifica o impacto da alteração nos demais requisitos e no sistema. Se é inviável, é descartada.

.Implementação da Alteração

- O documento de requisitos é alterado e, onde necessário, o projeto do sistema e sua implementação são alterados

Casos de Uso

Objetivos

.Principal saída da etapa de especificação

.Principal entrada da etapa de projeto

O que são?

- .Capturam os requisitos funcionais**
- .Descreve o sistema sob a perspectiva do usuário**
- .Descreve a interação típica do usuário com o sistema**

Características

- .São sempre iniciados por um ator**
- .Devem sempre retornar um resultado ao ator**
- .Cada caso de uso especifica uma funcionalidade completa envolvendo os atores interessados.**
- .Deve sempre terminar com o resultado que deve ser dado ao ator.**

Exemplo

.O cliente folheia o catálogo e seleciona itens para adicionar na cesta de compras. Quando o cliente deseja fechar a compra, deve informar os dados do cartão e confirmar a compra. O sistema autoriza a venda junto à operadora de cartão e notifica o cliente, tanto na tela, quanto via e-mail.

Cenários Derivados

- Nem sempre as coisas acontecem como esperamos
 - O cartão pode não ser autorizado
 - Um cliente conhecido pode pendurar a conta e só pagar no final do mês
- Todos esses cenários são diferentes, porém similares
 - Todos têm o mesmo objetivo: comprar produtos
 - Todos pertencem ao mesmo caso de uso: comprar produtos

Caso de Uso

- O caso de uso foca no cenário típico, onde tudo acontece na maior parte das vezes
- Os cenários alternativos também são descritos no caso de uso em uma seção separada
 - Também conhecido como extensões
 - Deve indicar o ponto em que estende o cenário típico
 - Deve indicar o ponto em que retorna ao cenário típico

Atores

•São as entidades que se relacionam com o sistema

•Exemplos

–Usuário

–Cliente

–Representante do cliente

–Gerente

–Sistema externo

–Etc.

Atores

- .Importante: ator é o papel, e não a pessoa
 - Se existe um cliente chamado João, existe um ator chamado “cliente” e não um ator chamado “João”
- .Um outro sistema que interage com o sistema a ser desenvolvido também é considerado um ator, desde que este sistema não faça parte do desenvolvimento

Identificação

- Leia a descrição em linguagem natural buscando por
 - Verbos: casos de uso candidatos
 - Substantivos: atores candidatos
- Exemplo
 - O cliente entra na loja para comprar produtos
- Cliente, loja e produtos são candidatos a atores (provavelmente loja é o sistema)
- Entrar e comprar são candidatos a casos de uso

Conteúdo do Caso de Uso

- Não existe consenso na estrutura interna de um caso de uso
- Algumas estruturas são utilizadas recorrentemente
 - Simples
 - Detalhada
- É importante selecionar uma estrutura que seja adequada para o problema e o processo em uso

Estrutura Simples

.Composta por

–Nome

–Cenário típico

–Cenários alternativos

Estrutura Simples - Exemplo

.Nome: UC1 – Compra de Produto

.Cenário Típico:

- 1. O cliente folheia o catálogo e seleciona itens para comprar**
- 2. O cliente fecha a compra**
- 3. O cliente escolhe a forma de entrega**
- 4. O sistema apresenta o preço total**
- 5. O cliente fornece os dados do cartão**
- 6. O sistema autoriza a compra**

Estrutura Simples - Exemplo

.Cenários Alternativos:

–5.a O cliente é conhecido

.1. Retorna para o passo 7 do fluxo principal

.2. O sistema pendura o pagamento

–6.a A operadora do cartão não autoriza a compra

.1. O cliente pode fornecer outro cartão, retornando para o passo 5, ou cancelar a compra.

Algumas definições

- **Fluxo Básico (típico):** Quando a atividade é realizada com sucesso. Deve existir somente um.
- **Fluxo Alternativo:** Quando a atividade não é realizada com sucesso. Podem existir vários.
- **Pré-condição:** condição necessária para iniciar o caso de uso.
- **Pós-condição:** tarefas que devem ser realizadas depois que as etapas do caso de uso tiverem sido concluídas

Estrutura Detalhada

- .Nome
- .Lista de Atores
- .Visão geral (descrição)
- .Referências Cruzadas
 - Requisitos
 - Outros casos relacionados
 - Classes que implementam o caso de uso
- .Gatilho

Estrutura Detalhada

- Lista de atores: Cliente, Sistema de autorização de cartão
- Visão geral: Um cliente acessa o site de venda de produtos e...
- Referências Cruzadas:
 - Requisitos: R1, R5, R10 (são os nomes dos requisitos)
 - Outros casos relacionados: UC3 – Login
 - Classes que implementam o caso de uso: Compra

Estrutura Detalhada

.Gatilho

- Não se aplica

.Pré-condições

- Antes de iniciar o caso de uso, o usuário deve fazer login no sistema (UC3 – Login)

.Pós-condições

- Caso a venda ocorra com sucesso, o estoque deve sofrer baixa da quantidade de produtos vendidos

Descrição de um cenário

- O cenário é composto de

- Uma lista de passos

- Uma tabela com colunas para atores e outra para o sistema

- Cada passo descreve

- O ator envolvido

- A sua intenção naquela interação

Cenário - Exemplo

Cliente	Sistema
1. Folheia o catálogo e seleciona itens para comprar	
2. Fecha a compra	
3. Escolhe forma de entrega	
	4. Apresenta preço total
5. Fornece dados do cartão	
	6. Autoriza a compra
	7. Confirma compra na tela
	8. Envia e-mail confirmando a compra

Perguntas para criar casos

- .Quem utiliza o sistema?
- .Como é o uso do sistema?
- .Quais informações são fornecidas ou obtidas do sistema?
- .Como o sistema é mantido?
- .Quais outros sistemas interagem com esse sistema?

Perguntas para criar casos

- Quando tudo dá certo, como o sistema se comporta?
- Algo pode acontecer diferente nesse passo?
 - Ex: operadora não autoriza compra
- O que pode dar errado nesse passo?
 - Ex: cliente fica sem conexão com internet

Caso de uso x Passo

- Quando um passo for muito complicado
 - Verifique se o cenário alternativo está se misturando com o cenário típico
 - Verifique se é possível transformar o passo em outro caso de uso
- Quando um passo vira um novo caso de uso
 - O primeiro passo deve incluir o segundo
 - Na descrição do caso de uso, use referência cruzada para o outro
- Cuidado! Não faça decomposição funcional do caso de uso!

Dicas

- Comecem da estrutura simples e adicionem somente os elementos necessários
- Façam casos de uso enxutos
 - Casos longos não são lidos
 - Prejudicam a abstração
- Adicionem detalhes de forma proporcional ao risco do caso de uso
 - Não detalhar em excesso os casos de uso de baixo risco

Diagramas de Casos de Uso

- A UML oferece um diagrama que permite visualizar
 - Os atores
 - Os casos de uso
 - O relacionamento entre atores e casos de uso
 - O relacionamento entre casos de uso
- A maior riqueza dos casos de uso está na sua descrição, não no diagrama

Elementos do Diagrama

.Ator



.Caso de Uso



.Participação de um Ator no caso de uso



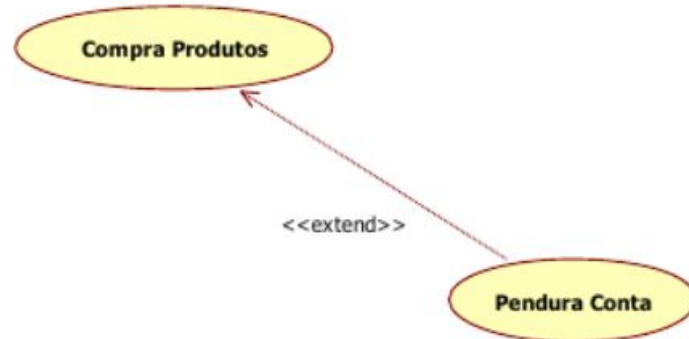
Relacionamento de Generalização

- Relação “é um” entre atores
- Relação “é um tipo de” entre casos de uso



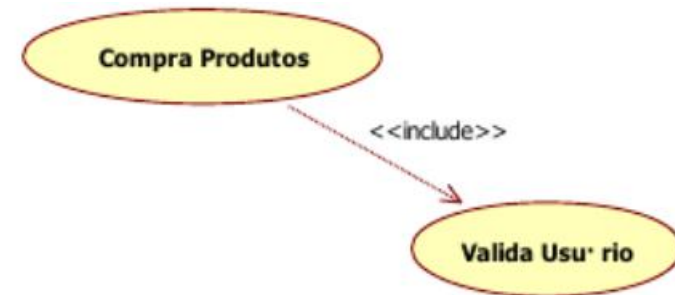
Relacionamento de Extensão

- Encapsula um cenário alternativo em outro caso de uso
- Utiliza o campo “gatilho” para definir o momento que entra em ação
- Se “Compra Produtos” for executado o “Pendura Conta” poderá ser executado (mas não obrigatoriamente)

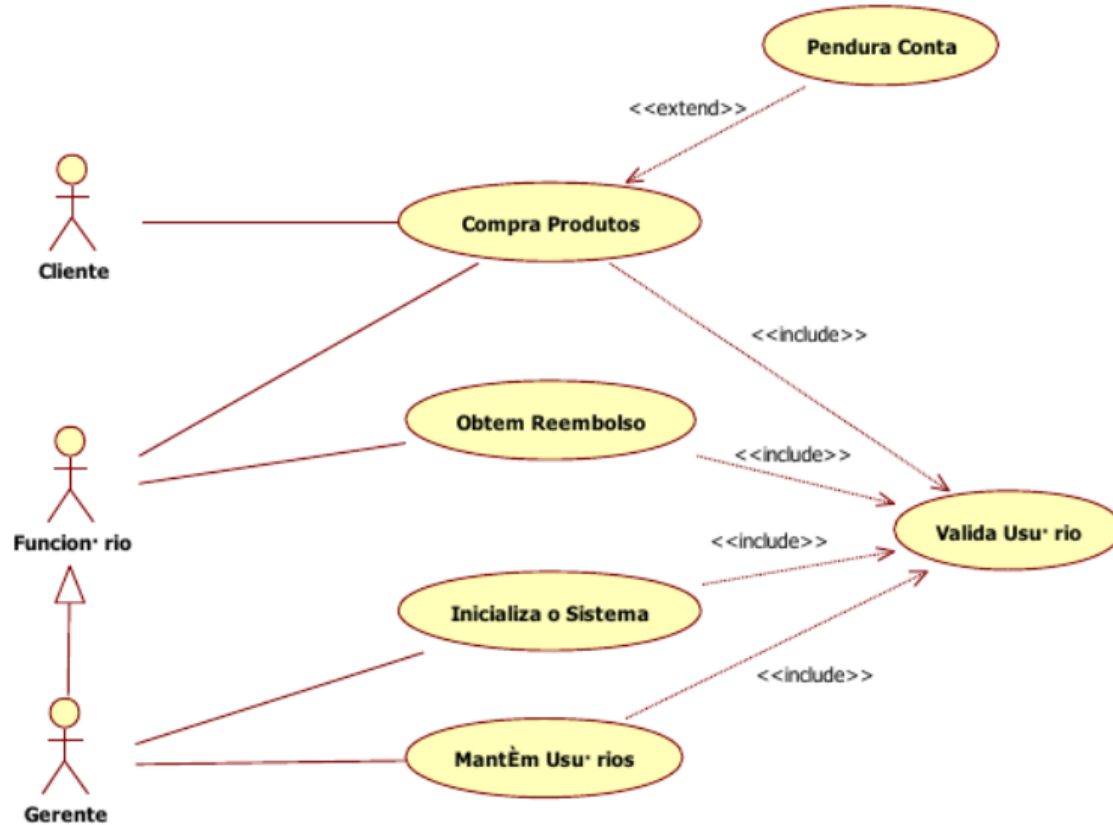


Relacionamento de Inclusão

- Adição de um comportamento em um ponto determinado do caso de uso
- Útil quando esse comportamento é repetido em vários casos do sistema
- “Compra Produto” inclui “Valida Usuário”
- Se “Compra Produto” então “Valida Usuário”



Exemplo



Exercício

- Elabora o diagrama e a descrição dos casos de uso de um sistema de ponto de vendas
- Sistema de Ponto de Vendas (Pode ser outro se quiserem)
 - Usualmente utilizado em caixas de supermercado ou lojas para permitir o controle das vendas
 - Pense em um estabelecimento comercial que você frequenta para poder enxergar os possíveis casos de uso

Construindo Casos de Uso

- Listar todas as funções/requisitos e defina o sistema
- Identificar atores e casos de uso
- Desenhe o diagrama de casos de uso
- Escreva todos os casos de uso em formato simples
- Escreva os casos mais críticos em formato detalhado

Construindo Casos de Uso

.Objetivo Geral:

- O propósito deste projeto é criar um terminal de ponto de vendas a ser usado em lojas de varejo para aumentar a automatização das compras

Construindo Casos de Uso

.Objetivos Específicos

- Checkout mais rápido para o cliente
- Verificação e identificação rápida do cliente
- Análise rápida e precisa do crédito

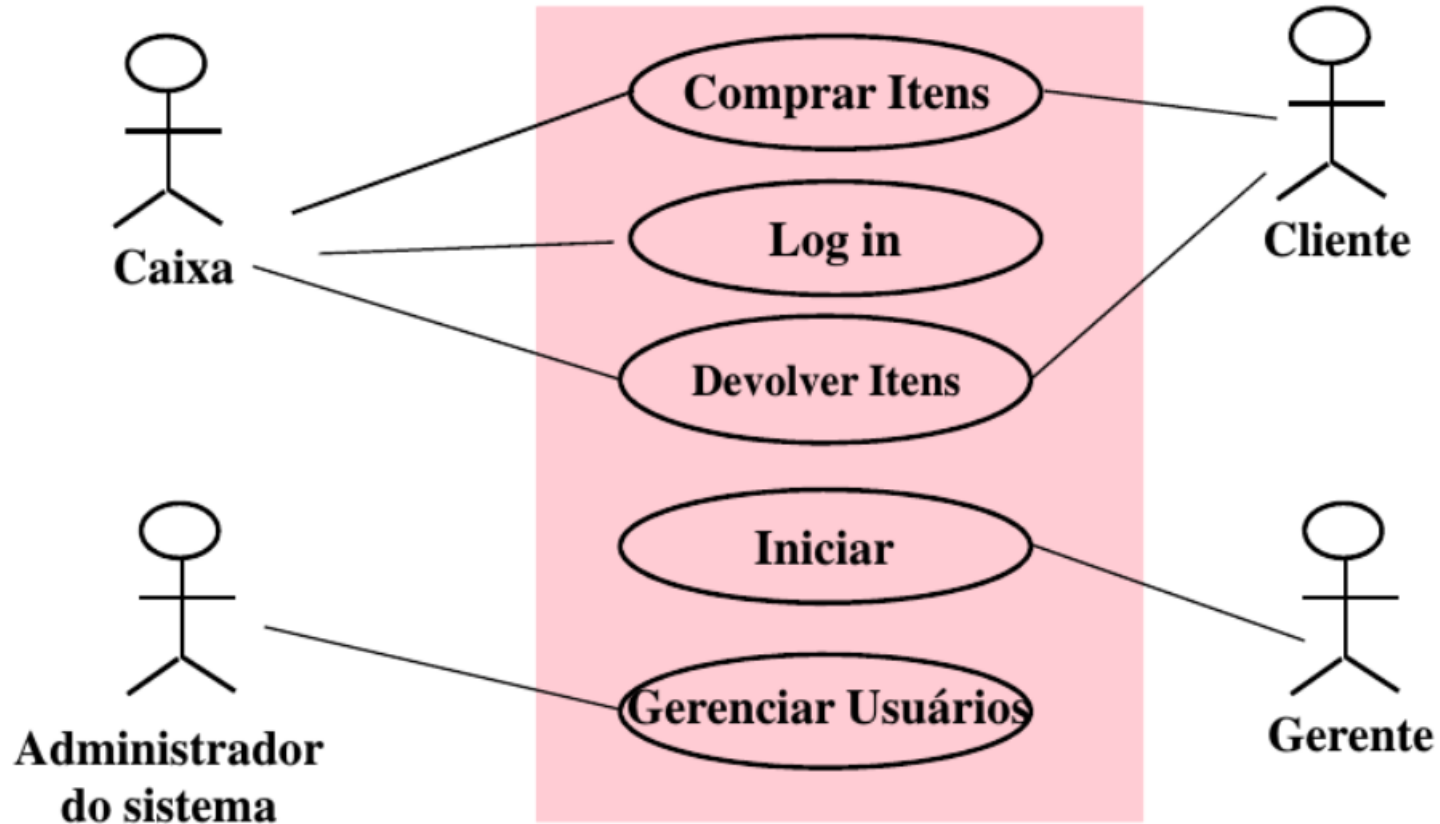
Passo 1.1: Listar requisitos

- O gerente deve abrir o caixa (login) com um identificador (ID) e uma senha para poder usar o sistema
- Tratar os pagamentos em dinheiro: capturar quantia recebida e informar o troco
- Tratar o pagamento por cartão: captar a informação do cartão por um leitor ou entrada manual e autorizar o pagamento com o serviço de autorização (externo) via internet

Passo 1.2: Listar todas as funções

- .Comprar itens**
- .Controlar estoque**
- .Gerenciar usuários**
- .Inicializar caixa**
- .Fazer login**
- .Efetuar pagamento com dinheiro**
- .Efetuar pagamento com cartão**

Passos 1, 2 e 3:



Passo 4: Casos de Uso Simples

- Caso de Uso: Comprar Itens

- Atores: Cliente, Caixa

- Descrição

- Um cliente chega a um ponto de pagamento, com vários itens que deseja comprar. O caixa registra os itens de compra e recebe o pagamento. No final, o cliente sai com os itens comprados

Passo 4: Casos de Uso Simples

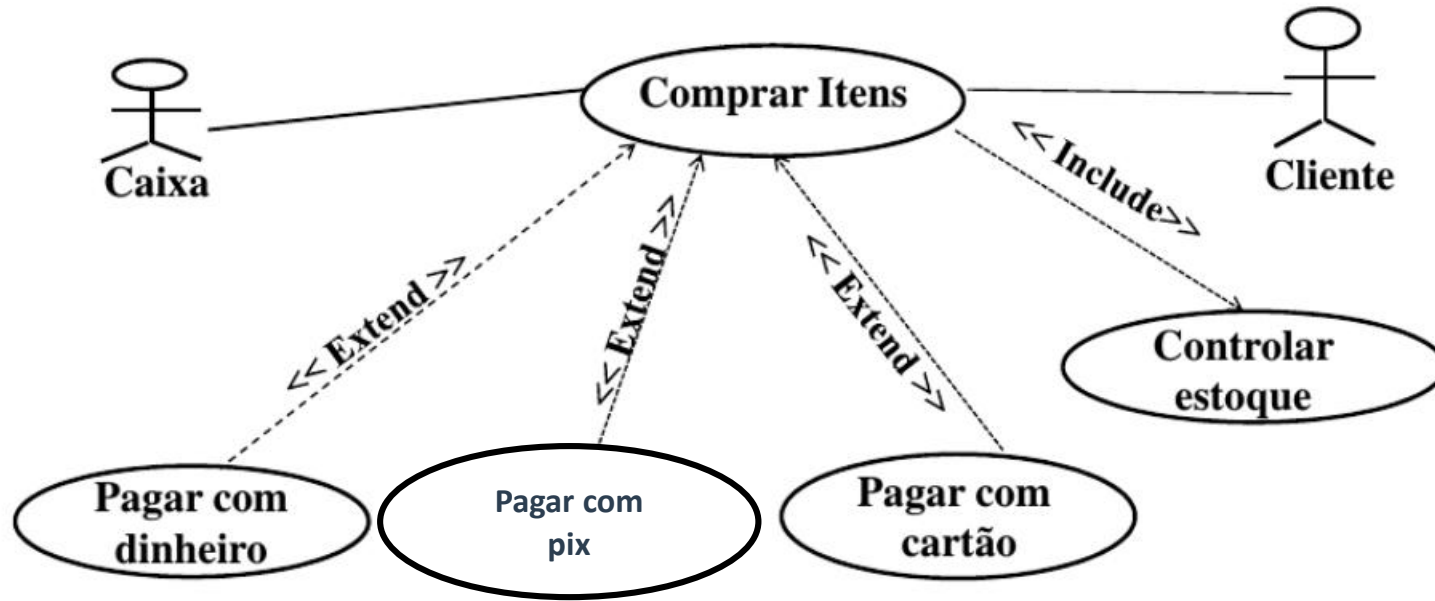
.Caso de Uso: Iniciar

.Atores: Gerente

.Descrição:

–O gerente liga o sistema de modo a prepará-lo para o uso pelos Caixas. O gerente verifica se a data e a hora estão corretas, em seguida o sistema está preparado para uso dos caixas

Caso de Uso essencial expandido



Paso 5.2: Sequência típica de eventos

Ator	Sistema
1. Cliente chega num ponto de pagamento com itens	
2. O caixa registra os itens. Se houver mais de um do mesmo item, o caixa pode entrar a quantidade.	3. Determina o preço do item e acrescenta informações sobre o item à transação de venda em andamento. Em seguida a descrição e o preço são apresentados.
4. Caixa indica que encerrou	5. Sistema apresenta o total
6. Caixa informa o total ao cliente	
7. Cliente informa o tipo de pagamento a) Se dinheiro ver UC-Pagar com dinheiro b) Se pix ver UC-Pagar com pix c) Se cartão ver UC-Pagar com cartão	

Continuação...

Ator	Sistema
	8. Registra a venda
	9. Atualiza o estoque, ver UC-Controlar Estoque
	10. Gera recibo
11. Dá recibo ao cliente e finaliza a compra	

Passo 5.3: Sequências Alternativas

.Linha 2 – Entrada de identificador de item inválido

–Indicar erro

.Linha 7 – Cliente não pode pagar

–Cancelar a transação

No Passo 5.2: UC – Pagar com dinheiro

Ator	Sistema
1. Cliente dá uma quantia em dinheiro, possivelmente maior que o total	
2. Caixa registra a quantia	3. Apresenta o valor do troco devido
4. Caixa deposita o dinheiro recebido, retira o troco devido e entrega ao cliente	

No Passo 5.3: Pagar com dinheiro

.Sequência Alternativa

–Linha 1 – O cliente não tem dinheiro suficiente.

.Seleciona outro método de pagamento ou cancela a transação

Vamos modelar outra coisa?

.Que tal um jogo?